Studio Nx

JS 107 — planSchedule()

2026-10-14

JS 107 — `planSchedule()` (스케줄러가 예약할 소리 목록 계산)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (새 파일, 소리와 시계를 쓰지 않는 스케줄러 계산 부분)
작업	S1-05 오디오 엔진 기초: 네 번째 함수 (FR-201 재생, FR-205 tick/PPQ 960, FR-202a 템포 변경)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	이 계산을 쓰는 실제 스케줄러 루프(`Transport`: AudioContext + setInterval lookahead), 그다음 메트로놈, 루프

1. 하는 일

브라우저에서 소리를 정확한 시각에 내려면 **lookahead 방식**을 씁니다. 짧은 타이머(약 25ms)가 자주 돌면서 "앞으로 0.1초 안에 울릴 소리"를 미리 `AudioContext` 시각으로 예약합니다. 이 함수는 그중에서 "지금 예약할 소리 목록"을 계산하는 **순수 계산** 부분입니다(소리·시계 없음 → Node로 시험 가능).

커서(tick, time)는 "아직 예약하지 않은 다음 소리"를 가리킵니다. `time`이 `untilTime`보다 작은 동안 커서를 `stepTicks`씩 옮기며 목록에 담고, 이어 쓸 커서(`nextTick`·`nextTime`)를 돌려줍니다.

입력	결과
커서 (0 tick, 0초), 1/16=240 tick, 120 BPM, until 0.1초	0.125초 간격으로 tick 0, 240, ..., 720 (8개)
커서 (3600 tick) 에서 변경점 3840(120→60)을 넘는 창	3840 뒤 단계의 간격이 0.125초 → 0.25초로 바뀜
untilTime ≤ fromTime	빈 목록, 커서 그대로

2. 코드

```
import { ticksToSecondsWithChanges } from './timing.js';

export const MAX_EVENTS_PER_PLAN = 100000;

export function planSchedule(fromTick, fromTime, untilTime, stepTicks, baseBpm, changes) {
  if (typeof fromTick !== 'number' || !isFinite(fromTick)) throw new RangeError('fromTick 은 유한한 숫자여야 합니다: ' + fromTick);
  if (typeof fromTime !== 'number' || !isFinite(fromTime)) throw new RangeError('fromTime 은 유한한 숫자여야 합니다: ' + fromTime);
  if (typeof untilTime !== 'number' || !isFinite(untilTime)) throw new RangeError('untilTime 은 유한한 숫자여야 합니다: ' + untilTime);
  if (typeof stepTicks !== 'number' || !isFinite(stepTicks) || stepTicks <= 0) throw new RangeError('stepTicks 는 0 보다 큰 숫자여야 합니다: ' + stepTicks);
  if (typeof baseBpm !== 'number' || !isFinite(baseBpm) || baseBpm <= 0) throw new RangeError('baseBpm 은 0 보다 큰 숫자여야 합니다: ' + baseBpm);

  const events = [];
  let tick = fromTick;
  let time = fromTime;
  while (time < untilTime) {
    if (events.length >= MAX_EVENTS_PER_PLAN) throw new RangeError('한 번에 예약할 단계가 너무 많습니다 ('
```

```
+ MAX_EVENTS_PER_PLAN + '개 초과)');
    events.push({ tick: tick, time: time });
    const nextTick = tick + stepTicks;
    const delta = ticksToSecondsWithChanges(nextTick, baseBpm, changes) - ticksToSecondsWithChanges(tic
k, baseBpm, changes);
    tick = nextTick;
    time += delta;
  }
  return { events: events, nextTick: tick, nextTime: time };
}
```

최신 문법으로 쓰면 (공부용)

```
export const planSchedule = (fromTick, fromTime, untilTime, stepTicks, baseBpm, changes) => {
  const events = [];
  let tick = fromTick;
  let time = fromTime;
  while (time < untilTime) {
    events.push({ tick, time });
    const nextTick = tick + stepTicks;
    time += ticksToSecondsWithChanges(nextTick, baseBpm, changes) - ticksToSecondsWithChanges(tick, base
Bpm, changes);
    tick = nextTick;
  }
  return { events, nextTick: tick, nextTime: time };
};
```

최신 문법	실제 코드	뜻
`{ tick, time }` (속기)	`{ tick: tick, time: time }`	이름이 같은 변수를 그대로 담습니다
`while …`	(같음)	실제 코드도 `while` 을 씁니다

3. 왜 이렇게

선택	이유
소리·시계와 분리한 순수 함수	소리 없이 Node로 계산을 시험할 수 있습니다. 소리를 내는 부분(`Transport`)은 이 결과를 받아 `AudioContext` 노드만 만듭니다
`time` 은 "절대 시각"으로 받음	커서 시각은 `AudioContext.currentTime + 여유` 같은 절대 시각입니다. 구간마다 더하는 값(차이)만 쓰므로 절대·상대 어느 쪽이어도 맞습니다
단계 초를 105의 차이로 구함	`ticksToSecondsWithChanges(t+step) - ticksToSecondsWithChanges(t)` 로 하면 한 단계가 템포 변경점을 걸쳐도 정확합니다(예: 3840을 넘는 단계)
커서를 돌려줌	스케줄러가 다음 타이머 때 이어서 쓰도록 합니다. 매번 처음부터 다시 계산하지 않습니다
`MAX_EVENTS_PER_PLAN` 안전 한계	`untilTime` 이 너무 멀거나 `stepTicks` 가 너무 작으면(잘못된 호출) 탭이 멈출 수 있습니다. 그때 오류로 알립니다. 정상 호출(0.1초 창)은 몇 개 안 됩니다
`baseBpm` 은 여기서도 검사	`untilTime ≤ fromTime` 이면 반복이 한 번도 안 돌아 105가 검사할 기회가 없습니다. 그래서 여기서도 검사합니다

4. 시험 (Node에서 직접 실행, 26개 통과)

확인	결과
120 BPM 1/16 창(0.1초): tick 0,240,… 간격, 시간 간격 0.125초	✓
모든 이벤트가 untilTime 미만, 커서의 time 은 untilTime 이상	✓
이벤트 time 이 `ticksToSecondsWithChanges` 직접 계산과 일치	✓
untilTime ≤ fromTime 이면 빈 목록, 커서 그대로	✓
연속 호출 시뮬레이션: 커서가 이어져 tick이 240씩 끊김 없이 증가, 시간 단조 증가	✓
변경점(STEP)을 걸친 창: 3840 뒤 간격이 0.125→0.25초	✓
LINEAR 곡에서도 tick이 240씩	✓
안전 한계 초과 시 `RangeError`, 값 오류 7종	✓